Reporte del proyecto: Algoritmos para el cálculo de todos los Testores Típicos de una matriz básica

Matemáticas Discretas — Semestre 202610

Integrantes:
Martín Enríquez (00347250), Paula Gallardo (00343921),
Eduardo Tapia (00346319)
Profesor: Eduardo Alba
Universidad San Francisco de Quito (USFQ)

Fecha de culminación: 8 de noviembre de 2026

Índice

1. Fundamentos teóricos **1**

2. Metodología e implementación computacional **2**
2.1. Lenguaje, estructura y arquitectura 2
2.2. Compilación y uso 4

3. Ejercicio 1: generación de matrices y matriz básica **4**
3.1. Generación 4
3.2. Reducción a matriz básica 4
3.3. Resultado para una matriz de 100×10 4

4. Ejercicio 2: algoritmo YYC **5**
4.1. Ejercicio 2.1: matriz básica A 5
4.2. El algoritmo YYC 6
4.3. Ejercicio 2.2: ejecución manual de YYC sobre A 6
4.4. Ejercicio 2.3: implementación computacional de YYC 9
4.5. Ejercicio 2.4: resultados sobre la matriz A 9

5. Ejercicio 3: algoritmo BT **10**
5.1. El algoritmo BT 10
5.2. Ejercicio 3.1: ejecución manual de BT sobre A 11
5.3. Ejercicio 3.2: implementación computacional de BT 12
5.4. Ejercicio 3.3: resultados sobre la matriz A 14

---

1. Fundamentos teóricos

En Reconocimiento de Patrones con enfoque lógico-combinatorio, los objetos de estudio se describen mediante un conjunto de rasgos (características) $R = \{x_1, \ldots, x_n\}$. Comparando, rasgo a rasgo, objetos que pertenecen a clases distintas se obtiene una *matriz de diferencias*

booleana: la entrada (i, j) vale 1 si el par de objetos i es distinguible por el rasgo x_j y 0 en caso contrario [3]. Toda la teoría de este proyecto trabaja sobre matrices booleanas de este tipo.

Definición 1 (Subfila y superfila)**.** Sean $p = (p_1, \ldots, p_n)$ y $t = (t_1, \ldots, t_n)$ dos filas de una matriz booleana. Decimos que p es *subfila* de t (y que t es *superfila* de p) si $p_j \leq t_j$ para todo j y $p \neq t$. Equivalentemente, el conjunto de columnas donde p tiene unos está estrictamente contenido en el de t.

Definición 2 (Matriz básica)**.** Una matriz booleana BM es *básica* si no tiene filas nulas, no tiene filas repetidas y ninguna de sus filas es superfila de otra. La *matriz básica* de una matriz M es la que se obtiene de M eliminando las filas nulas, dejando una sola copia de cada fila repetida y eliminando todas las superfilas.

Definición 3 (Densidad)**.** La densidad de una matriz booleana M de $m \times n$ es la proporción de entradas iguales a 1:

$$\rho(M) = \frac{\#\{(i, j) : M_{ij} = 1\}}{m \cdot n}.$$

Definición 4 (Testor y testor típico)**.** Un subconjunto de rasgos $T \subseteq R$ es un *testor* de M si la submatriz $M|_T$ (formada por las columnas de T) no contiene filas nulas, es decir, si toda fila de M tiene al menos un 1 en alguna columna de T. T es un *testor típico* si es testor y ningún subconjunto propio de T es testor.

Definición 5 (Fila típica y conjunto compatible)**.** Sea $T \subseteq R$ y $x_j \in T$. Una fila r es *fila típica* de x_j respecto de T si $r_j = 1$ y $r_k = 0$ para todo $x_k \in T \setminus \{x_j\}$. T es un *conjunto compatible* de M si cada rasgo de T tiene al menos una fila típica, es decir, si $M|_T$ contiene, salvo permutación de filas, a la matriz identidad de orden $|T|$ [1].

Proposición 1. *T es testor típico de M si y sólo si T es testor y conjunto compatible de M.*

Demostración. Ser testor es una propiedad monótona: si T es testor, todo superconjunto también lo es. Por tanto, T es típico si y sólo si $T \setminus \{x_j\}$ no es testor para cada $x_j \in T$. Que $T \setminus \{x_j\}$ no sea testor significa que existe una fila r que tiene sólo ceros en $T \setminus \{x_j\}$; como T sí es testor, $r_j = 1$. Esa fila es precisamente una fila típica de x_j. □

Proposición 2. *Una matriz M y su matriz básica BM tienen exactamente los mismos testores típicos (siempre que M no tenga filas nulas).*

Demostración. Si t es superfila de p, toda T que cubre a p cubre a t; luego eliminar t (o una copia repetida) no cambia el conjunto de testores. Como los testores típicos son los elementos minimales del conjunto de testores, tampoco cambian. Si M tiene una fila nula, ningún T la cubre y M no tiene testores; por eso en la reducción se descartan las filas nulas, que no aportan información de diferenciación. □

La Proposición 2 justifica que los algoritmos trabajen siempre sobre la matriz básica, que es (en general) mucho más pequeña.

2. Metodología e implementación computacional

2.1. Lenguaje, estructura y arquitectura

La implementación se realizó en C++17 y se compila con CMake. El código se organizó siguiendo una *arquitectura limpia* de cuatro capas, cada una compilada como una librería independiente; así, el propio sistema de compilación impide que una capa interna dependa de una externa:

Capa	Responsabilidad
domain	Conceptos matemáticos puros: `ColumnSet` (conjunto de columnas como mapa de bits), `BooleanMatrix`, reducción a matriz básica, definiciones de testor, fila típica y conjunto compatible, y los algoritmos `YycSolver` y `BtSolver`. No realiza entrada/salida.
application	Casos de uso: generar matrices, buscar una matriz básica que cumpla restricciones, ejecutar YYC midiendo el tiempo acumulado por fila y ejecutar BT midiendo su tiempo total. Define *puertos* (interfaces) para el azar, el reloj y la notificación de progreso.
infrastructure	Implementaciones concretas de los puertos: generador Mersenne Twister de 64 bits con semilla, reloj monótono, lectura/escritura de matrices y exportación de tablas a CSV y LaTeX.
presentation	Interfaz de usuario: menú interactivo y subcomandos de línea de comandos.

Representación. Cada fila de la matriz y cada conjunto de rasgos se representa con un `ColumnSet`: un arreglo fijo de ocho palabras de 64 bits (hasta 512 columnas). Las operaciones "T cubre a la fila r" ($T \cap r \neq \emptyset$) y "$T \cap r$ tiene exactamente un elemento" se resuelven con operaciones `AND` y conteo de bits sobre palabras completas. Un primer diseño guardaba las palabras en memoria dinámica; cambiar a almacenamiento fijo redujo el tiempo de YYC sobre la matriz A de unos 88 μs a unos 7 μs, porque el ciclo interno crea muchos conjuntos temporales.

Seguridad y robustez.

- Toda entrada numérica se valida de forma estricta (rango y cadena completa); una entrada inválida vuelve a preguntarse en lugar de dejar el flujo en error.
- Los archivos de matriz se validan carácter por carácter (sólo `0`, `1` y separadores) y se informa la línea del error; se limita el tamaño del archivo (16 MiB) y de la matriz (10^5 filas, 512 columnas).
- YYC tiene una cota configurable sobre $|\Psi^*|$ (2 millones de conjuntos, unos 160 MB) para no agotar la memoria con matrices patológicas.
- No se sobrescribe ningún archivo existente sin autorización explícita (`-forzar` o confirmación en el menú).
- Se compila con todas las advertencias activadas (`-Wall -Wextra -Wconversion...`) sin ninguna advertencia, y con protecciones del compilador (`-fstack-protector-strong`, `_FORTIFY_SOURCE`, `_GLIBCXX_ASSERTIONS`).

Verificación. El proyecto incluye 63 pruebas unitarias. La más importante compara el resultado de YYC con un *oráculo exhaustivo* independiente (recorre los 2^n subconjuntos de columnas y aplica directamente la definición de testor típico) sobre 400 matrices básicas aleatorias de hasta 12 columnas, y además comprueba que, después de cada fila, Ψ^* coincide exactamente con los testores típicos de las filas procesadas. Las pruebas de BT se describen en la Sección 5.3.

Reproducibilidad. Toda generación aleatoria usa una semilla que el programa informa. El bit aleatorio se obtiene como $u < p$ con $u \in [0, 1)$ formado por los 53 bits altos de `mt19937_64`, cuya secuencia está fijada por el estándar de C++; así, la misma semilla produce la misma matriz con cualquier compilador. El script `scripts/generar_resultados.sh` regenera todos los datos y tablas de este reporte.

2.2. Compilación y uso

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
ctest --test-dir build                # pruebas
./build/testores                      # menú interactivo
./build/testores ayuda                # subcomandos disponibles
```

Entorno experimental. Todos los tiempos se midieron en un equipo con procesador Intel Core i7-1265U, 16 GB de RAM, Windows 11 Pro, compilador GCC 16.1 (MSYS2/UCRT64) con optimización `-O3` (`Release`). El reloj monótono de Windows tiene una resolución de 100 ns.

3. Ejercicio 1: generación de matrices y matriz básica

Enunciado. Desarrollar un programa que genere matrices booleanas aleatorias de dimensiones dadas (al menos hasta 100×10), que transforme cada matriz en su matriz básica y calcule la densidad de esta última.

3.1. Generación

El usuario indica el número de filas $m \in [1, 10^5]$, de columnas $n \in [1, 512]$, la probabilidad p de que una entrada sea 1 (por defecto $p = 0.5$) y, opcionalmente, una semilla. Cada entrada se genera de forma independiente como una variable de Bernoulli de parámetro p.

3.2. Reducción a matriz básica

El procedimiento `reduceToBasic` aplica la definición en dos pasos y, además de la matriz básica, devuelve para cada fila original la razón por la que se conservó o eliminó:

1. Recorre las filas en orden; descarta las filas nulas y las que son idénticas a una fila anterior ya aceptada (se conserva la primera aparición).
2. Entre las filas restantes, conserva exactamente las *minimales*: aquellas que no tienen ninguna subfila. Cada superfila eliminada se justifica con una subfila suya que sí permanece en la matriz básica (existe por transitividad de la relación de subfila, que es un orden parcial estricto sobre filas distintas).

El orden relativo de las filas que permanecen no cambia y el número de columnas tampoco. El costo es $O(m^2\lceil n/64 \rceil)$. Finalmente la densidad se calcula contando los unos con conteo de bits por palabra.

3.3. Resultado para una matriz de 100×10

Con `testores generar -filas 100 -columnas 10 -semilla 2026` se obtuvo una matriz aleatoria de densidad 0.51. La reducción eliminó 5 filas repetidas, 0 nulas y 82 superfilas, y

produjo la siguiente matriz básica de 13×10 (archivo `data/ej1_basica_100x10.txt`):

$$BM = \begin{pmatrix} 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}, \qquad \rho(BM) = \frac{39}{13 \cdot 10} = 0.30.$$

Se observa un fenómeno general: la matriz básica tiene una densidad menor que la original (0.30 frente a 0.51), porque la reducción elimina precisamente las filas con más unos (las superfilas). Las filas con un único 1 (la de x_2 y la de x_7) obligan a que esos rasgos pertenezcan a todo testor de la matriz.

4. Ejercicio 2: algoritmo YYC

4.1. Ejercicio 2.1: matriz básica A

Enunciado. Generar una matriz básica A con al menos siete columnas y densidad mayor o igual a 0.4.

Como la reducción tiende a bajar la densidad, se implementó el caso de uso `searchBasicMatrix`: genera matrices aleatorias con una semilla fija hasta que la matriz básica cumpla las restricciones (densidad mínima, rango de filas y que ninguna columna quede nula). Se pidió una matriz básica de 7 columnas y 6 filas (tamaño adecuado para la ejecución manual):

```
testores buscar --columnas 7 --filas 8 --densidad-min 0.4 \
   --min-filas-basica 6 --max-filas-basica 6 --semilla 2026 --salida data/matriz_A.txt
```

El primer intento ya cumplió las condiciones. La matriz aleatoria generada (izquierda) tiene densidad $31/56 \approx 0.5536$; la fila r_2 es superfila de $r_5 = (0,0,1,1,0,1,1)$ y r_6 también es superfila de r_5, por lo que ambas se eliminan. El resultado es la matriz básica A (derecha):

$$M = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 \end{pmatrix} \quad \longrightarrow \quad A = \begin{array}{c} \begin{matrix} x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 \end{matrix} \\ \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 \end{pmatrix} \begin{matrix} r_1 \\ r_2 \\ r_3 \\ r_4 \\ r_5 \\ r_6 \end{matrix} \end{array}$$

A tiene 20 unos en $6 \cdot 7 = 42$ entradas, luego $\rho(A) = 20/42 \approx \mathbf{0.4762} \geq 0.4$, y tiene 7 columnas, todas con al menos un 1. Se verifica que es básica: no hay filas nulas ni repetidas, y para cada par de filas distintas cada una tiene un 1 donde la otra tiene 0.

4.2. El algoritmo YYC

YYC [1] es un algoritmo *incremental*: procesa las filas de la matriz básica una por una y mantiene el conjunto Ψ^* de los testores típicos de la submatriz formada por las filas ya procesadas.

1. **Inicialización.** $\Psi^* \leftarrow \{\{x_j\} : r_1[j] = 1\}$.
2. **Para cada fila** r_k, $k = 2, \ldots, m$: $\Psi_{aux} \leftarrow \emptyset$ y, para cada $\tau \in \Psi^*$:
   - si $\tau \cap r_k \neq \emptyset$ (τ ya cubre la fila), τ pasa a Ψ_{aux} sin cambios;
   - si no, para cada rasgo x con $r_k[x] = 1$, si $\tau \cup \{x\}$ es un conjunto compatible de las filas $r_1, \ldots, r_k$, se agrega a Ψ_{aux}.

   Al terminar, $\Psi^* \leftarrow \Psi_{aux}$.
3. Al procesar la última fila, Ψ^* es el conjunto de todos los testores típicos de la matriz.

Por qué funciona. Por inducción, supongamos que Ψ^* contiene los testores típicos de las filas $r_1, \ldots, r_{k-1}$. Todo elemento de Ψ_{aux} cubre r_k y las filas anteriores, por lo que es testor de las primeras k filas; además se exige que sea compatible, así que por la Proposición 1 es típico. Recíprocamente, si T es típico para las primeras k filas, o bien ya era típico antes (y cubre r_k), o bien $T = \tau \cup \{x\}$ donde r_k es la fila típica de x y τ era típico para las filas anteriores. Por último, no se generan duplicados: si $\tau_1 \cup \{x\} = \tau_2 \cup \{y\}$ con $x \neq y$, entonces $y \in \tau_1$, pero $y \in r_k$ y τ_1 no corta a r_k, lo cual es una contradicción.

Observación útil para el cálculo manual. Al extender un τ que no corta a r_k con un $x \in r_k$, la propia fila r_k es fila típica de x en $\tau \cup \{x\}$ (pues r_k tiene ceros en todo τ). Por lo tanto basta revisar que *cada rasgo de τ conserve alguna fila típica*, es decir, que tenga una fila donde aparece sin x ni otros rasgos de τ. Agregar x sólo puede "romper" filas típicas de τ que tengan un 1 en la columna x.

4.3. Ejercicio 2.2: ejecución manual de YYC sobre A

Escribimos cada fila de A como el conjunto de rasgos donde tiene un 1:

$$r_1 = \{x_1, x_3, x_5, x_7\}, \quad r_2 = \{x_1, x_5, x_6\}, \quad r_3 = \{x_1, x_2, x_4\},$$
$$r_4 = \{x_3, x_4, x_6, x_7\}, \quad r_5 = \{x_1, x_4, x_6\}, \quad r_6 = \{x_1, x_4, x_7\}.$$

En cada paso se recorre Ψ^* en el orden en que fue construido. Para cada candidato $\tau \cup \{x\}$ se escribe la intersección de cada fila procesada con el candidato; una intersección con un único elemento $\{x_j\}$ es una fila típica de x_j. El símbolo ✓ indica que el candidato es compatible y se agrega; × indica que se descarta y se señala el rasgo que se queda sin fila típica.

Paso 1: fila $r_1 = \{x_1, x_3, x_5, x_7\}$

Inicialización: cada rasgo presente en r_1 forma un conjunto unitario.

$$\Psi^* = \big\{\{x_1\},\ \{x_3\},\ \{x_5\},\ \{x_7\}\big\} \qquad |\Psi^*| = 4.$$

Paso 2: fila $r_2 = \{x_1, x_5, x_6\}$

- $\tau = \{x_1\}$: $\tau \cap r_2 = \{x_1\} \neq \emptyset \Rightarrow$ se conserva.
- $\tau = \{x_3\}$: no cubre r_2; se prueba con $x \in \{x_1, x_5, x_6\}$:
  - $\{x_1, x_3\}$: $r_1 \to \{x_1, x_3\}$, $r_2 \to \{x_1\}$. x_3 sólo aparece en r_1 junto a x_1: × (x_3 sin fila típica).
  - $\{x_3, x_5\}$: $r_1 \to \{x_3, x_5\}$, $r_2 \to \{x_5\}$: × (x_3 sin fila típica).
  - $\{x_3, x_6\}$: $r_1 \to \{x_3\}$, $r_2 \to \{x_6\}$: ✓.

- $\tau = \{x_5\}$: $\tau \cap r_2 = \{x_5\} \Rightarrow$ se conserva.
- $\tau = \{x_7\}$: no cubre r_2:
  - $\{x_1, x_7\}$: $r_1 \to \{x_1, x_7\}$, $r_2 \to \{x_1\}$: $\times$ (x_7 sin fila típica).
  - $\{x_5, x_7\}$: $r_1 \to \{x_5, x_7\}$, $r_2 \to \{x_5\}$: $\times$ (x_7 sin fila típica).
  - $\{x_6, x_7\}$: $r_1 \to \{x_7\}$, $r_2 \to \{x_6\}$: ✓.

$$\Psi^* = \big\{\{x_1\},\ \{x_3, x_6\},\ \{x_5\},\ \{x_6, x_7\}\big\} \qquad |\Psi^*| = 4.$$

Paso 3: fila $r_3 = \{x_1, x_2, x_4\}$

- $\tau = \{x_1\}$: cubre $r_3 \Rightarrow$ se conserva.
- $\tau = \{x_3, x_6\}$: no cubre r_3:
  - $\{x_1, x_3, x_6\}$: $r_1 \to \{x_1, x_3\}$, $r_2 \to \{x_1, x_6\}$, $r_3 \to \{x_1\}$: $\times$ (x_3 y x_6 pierden sus filas típicas r_1 y r_2, que contienen a x_1).
  - $\{x_2, x_3, x_6\}$: $r_1 \to \{x_3\}$, $r_2 \to \{x_6\}$, $r_3 \to \{x_2\}$: ✓.
  - $\{x_3, x_4, x_6\}$: $r_1 \to \{x_3\}$, $r_2 \to \{x_6\}$, $r_3 \to \{x_4\}$: ✓.
- $\tau = \{x_5\}$: no cubre r_3:
  - $\{x_1, x_5\}$: $r_1 \to \{x_1, x_5\}$, $r_2 \to \{x_1, x_5\}$, $r_3 \to \{x_1\}$: $\times$ (x_5 sin fila típica).
  - $\{x_2, x_5\}$: $r_1 \to \{x_5\}$, $r_2 \to \{x_5\}$, $r_3 \to \{x_2\}$: ✓.
  - $\{x_4, x_5\}$: $r_1 \to \{x_5\}$, $r_2 \to \{x_5\}$, $r_3 \to \{x_4\}$: ✓.
- $\tau = \{x_6, x_7\}$: no cubre r_3:
  - $\{x_1, x_6, x_7\}$: $r_1 \to \{x_1, x_7\}$, $r_2 \to \{x_1, x_6\}$, $r_3 \to \{x_1\}$: $\times$ (x_6 y x_7 sin fila típica).
  - $\{x_2, x_6, x_7\}$: $r_1 \to \{x_7\}$, $r_2 \to \{x_6\}$, $r_3 \to \{x_2\}$: ✓.
  - $\{x_4, x_6, x_7\}$: $r_1 \to \{x_7\}$, $r_2 \to \{x_6\}$, $r_3 \to \{x_4\}$: ✓.

$$\Psi^* = \big\{\{x_1\},\ \{x_2, x_3, x_6\},\ \{x_3, x_4, x_6\},\ \{x_2, x_5\},\ \{x_4, x_5\},\ \{x_2, x_6, x_7\},\ \{x_4, x_6, x_7\}\big\} \qquad |\Psi^*| = 7.$$

Paso 4: fila $r_4 = \{x_3, x_4, x_6, x_7\}$

- $\tau = \{x_1\}$: no cubre r_4. Respecto de $\{x_1\}$, las filas r_1, r_2, r_3 son filas típicas de x_1; al agregar x basta que alguna de ellas no contenga a x:
  - $\{x_1, x_3\}$: $r_2 \to \{x_1\}$, $r_4 \to \{x_3\}$: ✓.
  - $\{x_1, x_4\}$: $r_1 \to \{x_1\}$, $r_4 \to \{x_4\}$: ✓.
  - $\{x_1, x_6\}$: $r_1 \to \{x_1\}$, $r_4 \to \{x_6\}$: ✓.
  - $\{x_1, x_7\}$: $r_2 \to \{x_1\}$, $r_4 \to \{x_7\}$: ✓.
- $\tau = \{x_2, x_3, x_6\}$: $\tau \cap r_4 = \{x_3, x_6\} \Rightarrow$ se conserva.
- $\tau = \{x_3, x_4, x_6\}$: $\tau \cap r_4 = \{x_3, x_4, x_6\} \Rightarrow$ se conserva.
- $\tau = \{x_2, x_5\}$: no cubre r_4 (sus filas típicas: r_3 para x_2; r_1, r_2 para x_5):
  - $\{x_2, x_3, x_5\}$: $r_2 \to \{x_5\}$, $r_3 \to \{x_2\}$, $r_4 \to \{x_3\}$: ✓.
  - $\{x_2, x_4, x_5\}$: $r_3 \to \{x_2, x_4\}$ y x_2 no aparece en otra fila: $\times$ (x_2 sin fila típica).
  - $\{x_2, x_5, x_6\}$: $r_1 \to \{x_5\}$, $r_3 \to \{x_2\}$, $r_4 \to \{x_6\}$: ✓.
  - $\{x_2, x_5, x_7\}$: $r_2 \to \{x_5\}$, $r_3 \to \{x_2\}$, $r_4 \to \{x_7\}$: ✓.
- $\tau = \{x_4, x_5\}$: $\tau \cap r_4 = \{x_4\} \Rightarrow$ se conserva.
- $\tau = \{x_2, x_6, x_7\}$: $\tau \cap r_4 = \{x_6, x_7\} \Rightarrow$ se conserva.
- $\tau = \{x_4, x_6, x_7\}$: $\tau \cap r_4 = \{x_4, x_6, x_7\} \Rightarrow$ se conserva.

$$\Psi^* = \big\{\{x_1, x_3\},\ \{x_1, x_4\},\ \{x_1, x_6\},\ \{x_1, x_7\},\ \{x_2, x_3, x_6\},\ \{x_3, x_4, x_6\},\ \{x_2, x_3, x_5\},\ \{x_2, x_5, x_6\},\ \{x_2, x_5, x_7\},\ \{x_4, x_5\},\ \{x_2, x_6, x_7\},\ \{x_4, x_6, x_7\}\big\}$$
$$|\Psi^*| = 12.$$

Paso 5: fila $r_5 = \{x_1, x_4, x_6\}$

Diez de los doce conjuntos cortan a r_5 y se conservan: los cuatro que contienen x_1, los que contienen x_6 ($\{x_2, x_3, x_6\}$, $\{x_3, x_4, x_6\}$, $\{x_2, x_5, x_6\}$, $\{x_2, x_6, x_7\}$, $\{x_4, x_6, x_7\}$) y $\{x_4, x_5\}$. Sólo

hay que extender dos:

- $\tau = \{x_2, x_3, x_5\}$: la única fila donde aparece x_2 es $r_3 = \{x_1, x_2, x_4\}$.
  - $\{x_1, x_2, x_3, x_5\}$: $r_3 \to \{x_1, x_2\}$: × (x_2 sin fila típica).
  - $\{x_2, x_3, x_4, x_5\}$: $r_3 \to \{x_2, x_4\}$: × (x_2 sin fila típica).
  - $\{x_2, x_3, x_5, x_6\}$: $r_1 \to \{x_3, x_5\}$, $r_2 \to \{x_5, x_6\}$, $r_4 \to \{x_3, x_6\}$: × (x_3 y x_5 sin fila típica).
- $\tau = \{x_2, x_5, x_7\}$:
  - $\{x_1, x_2, x_5, x_7\}$ y $\{x_2, x_4, x_5, x_7\}$: igual que antes, r_3 deja de ser típica de x_2: ×.
  - $\{x_2, x_5, x_6, x_7\}$: $r_1 \to \{x_5, x_7\}$, $r_2 \to \{x_5, x_6\}$: × (x_5 sin fila típica).

Ninguna extensión es compatible, así que $\{x_2, x_3, x_5\}$ y $\{x_2, x_5, x_7\}$ desaparecen:

$$\Psi^* = \big\{\{x_1, x_3\},\ \{x_1, x_4\},\ \{x_1, x_6\},\ \{x_1, x_7\},\ \{x_2, x_3, x_6\},\ \{x_3, x_4, x_6\},\ \{x_2, x_5, x_6\},\ \{x_4, x_5\},\ \{x_2, x_6, x_7\},\ \{x_4, x_6, x_7\}\big\} \qquad |\Psi^*| = 10.$$

Paso 6: fila $r_6 = \{x_1, x_4, x_7\}$

Cortan a r_6 y se conservan: $\{x_1, x_3\}$, $\{x_1, x_4\}$, $\{x_1, x_6\}$, $\{x_1, x_7\}$, $\{x_3, x_4, x_6\}$, $\{x_4, x_5\}$, $\{x_2, x_6, x_7\}$ y $\{x_4, x_6, x_7\}$. No cortan a r_6:

- $\tau = \{x_2, x_3, x_6\}$: con $x = x_1$ o $x = x_4$ la fila r_3 deja de ser típica de x_2 (×); con $x = x_7$, $\{x_2, x_3, x_6, x_7\}$: $r_1 \to \{x_3, x_7\}$, $r_4 \to \{x_3, x_6, x_7\}$, así que x_3 queda sin fila típica (×).
- $\tau = \{x_2, x_5, x_6\}$: con $x = x_1$ o $x = x_4$, x_2 pierde r_3 (×); con $x = x_7$, $\{x_2, x_5, x_6, x_7\}$: $r_1 \to \{x_5, x_7\}$, $r_2 \to \{x_5, x_6\}$, así que x_5 queda sin fila típica (×).

$$\boxed{\Psi^* = \big\{\{x_1, x_3\},\ \{x_1, x_4\},\ \{x_1, x_6\},\ \{x_1, x_7\},\ \{x_4, x_5\},\ \{x_2, x_6, x_7\},\ \{x_3, x_4, x_6\},\ \{x_4, x_6, x_7\}\big\}} \qquad |\Psi^*| = 8.$$

Comprobación. La Tabla 1 verifica, para cada uno de los 8 testores típicos, que es testor (ninguna fila queda con intersección vacía) y que cada rasgo tiene una fila típica.

T	$r_1 \cap T$	$r_2 \cap T$	$r_3 \cap T$	$r_4 \cap T$	$r_5 \cap T$	$r_6 \cap T$	Filas típicas
$\{x_1, x_3\}$	x_1x_3	x_1	x_1	x_3	x_1	x_1	x_1:r_2, x_3:r_4
$\{x_1, x_4\}$	x_1	x_1	x_1x_4	x_4	x_1x_4	x_1x_4	x_1:r_1, x_4:r_4
$\{x_1, x_6\}$	x_1	x_1x_6	x_1	x_6	x_1x_6	x_1	x_1:r_1, x_6:r_4
$\{x_1, x_7\}$	x_1x_7	x_1	x_1	x_7	x_1	x_1x_7	x_1:r_2, x_7:r_4
$\{x_4, x_5\}$	x_5	x_5	x_4	x_4	x_4	x_4	x_4:r_3, x_5:r_1
$\{x_2, x_6, x_7\}$	x_7	x_6	x_2	x_6x_7	x_6	x_7	x_2:r_3, x_6:r_2, x_7:r_1
$\{x_3, x_4, x_6\}$	x_3	x_6	x_4	$x_3x_4x_6$	x_4x_6	x_4	x_3:r_1, x_4:r_3, x_6:r_2
$\{x_4, x_6, x_7\}$	x_7	x_6	x_4	$x_4x_6x_7$	x_4x_6	x_4x_7	x_4:r_3, x_6:r_2, x_7:r_1

Tabla 1: Verificación de los testores típicos de A: todas las intersecciones son no vacías (testor) y cada rasgo tiene fila típica (compatible).

Observaciones de la ejecución:

- $|\Psi^*|$ no es monótono: pasa por $4, 4, 7, 12, 10, 8$. Las filas r_5 y r_6 eliminan candidatos, porque ningún conjunto que no las cubre admite una extensión compatible.
- Las extensiones se descartan casi siempre por la misma razón: el único rasgo x disponible en la nueva fila aparece también en la única fila típica de algún rasgo de τ (por ejemplo, x_2 sólo tiene a r_3, que también contiene a x_1 y x_4).
- En total se evaluaron $6+9+8+6+6 = 35$ candidatos y $2+6+7 = 15$ fueron compatibles; nunca se examinó un subconjunto que no fuera extensión directa de un elemento de Ψ^*, mientras que una búsqueda exhaustiva examinaría $2^7 - 1 = 127$ subconjuntos.

4.4. Ejercicio 2.3: implementación computacional de YYC

Diseño. El algoritmo vive en la clase de dominio `YycSolver`, que expone una interfaz incremental: `processNextRow()` procesa una fila y `current()` devuelve Ψ^*. Esto permite que el caso de uso `runYyc` de la capa de aplicación:

1. lea el reloj monótono antes y después de `processNextRow()` y sume sólo ese intervalo al **tiempo acumulado**;
2. después de cada fila, y *fuera* del intervalo medido, notifique a un *listener* con Ψ^* y el tiempo acumulado. La presentación muestra entonces los testores encontrados hasta ese momento (requisitos 1 y 2 del enunciado) sin que imprimir en pantalla contamine la medición.

La comprobación de compatibilidad recorre las filas $r_1, \ldots, r_k$, calcula $r_i \cap T$ con operaciones de bits y marca los rasgos que aparecen solos; T es compatible si todos quedan marcados. Opcionalmente, el solver registra una traza con cada decisión (qué τ se conservó, qué extensiones se probaron y qué rasgo se quedó sin fila típica); con ella se generó la explicación paso a paso usada para validar la sección anterior.

Interfaz de usuario. El programa `testores` ofrece un menú interactivo (generar, buscar, cargar desde archivo, ingresar a mano, ver la matriz básica y su densidad, ejecutar YYC normal o paso a paso, guardar y exportar resultados) y, para automatizar, subcomandos:

```
testores yyc --entrada data/matriz_A.txt              # Psi* y tiempo tras cada fila
testores yyc --entrada data/matriz_A.txt --traza      # explicación paso a paso
testores yyc --entrada data/matriz_A.txt --repeticiones 10000 \
             --csv resultados/yyc_A.csv --latex resultados/yyc_A_tabla.tex
```

Si la matriz introducida no es básica, el programa lo informa, la reduce (Proposición 2) y aplica YYC a la matriz básica. Un fragmento de la salida (con `Psi*` en lugar del símbolo griego) es:

```
Tras procesar r3 (3/6): |Psi*| = 7 | tiempo acumulado = 12.500 us
  Psi* = {x1}, {x2, x5}, {x4, x5}, {x2, x3, x6}, {x2, x6, x7}, {x3, x4, x6}, {x4, x6, x7}
Tras procesar r4 (4/6): |Psi*| = 12 | tiempo acumulado = 16.700 us
  ...
```

4.5. Ejercicio 2.4: resultados sobre la matriz A

Una sola ejecución de YYC sobre A dura unos pocos microsegundos, una escala en la que el ruido del sistema operativo (interrupciones, caché fría, cambios de frecuencia) es comparable al propio tiempo medido; en la primera ejecución, por ejemplo, la fila r_1 tardó $5.6\,\mu$s porque la caché aún estaba fría. Por eso la tabla reporta, para cada fila, la **mediana** de su tiempo de procesamiento en 10 000 ejecuciones, acumulada fila a fila (lo que garantiza que la columna sea creciente). Los conjuntos de testores son idénticos en todas las ejecuciones y coinciden con la ejecución manual.

Interpretación.

- El conjunto final coincide exactamente con el obtenido a mano (8 testores típicos: 5 de cardinalidad 2 y 3 de cardinalidad 3).
- El tiempo por fila crece con el trabajo realizado en ella. La fila r_4, que genera 8 candidatos y deja $|\Psi^*| = 12$, y las filas r_5 y r_6, que deben recorrer Ψ^* completo y probar 6 extensiones cada una contra 5 y 6 filas, son las más costosas. La fila r_1 sólo crea los conjuntos unitarios.

Fila	$\lvert\Psi^*\rvert$	Testores típicos de las filas procesadas	T. acum. (μs)
r_1	4	$\{x_1\}$, $\{x_3\}$, $\{x_5\}$, $\{x_7\}$	0.500
r_2	4	$\{x_1\}$, $\{x_5\}$, $\{x_3, x_6\}$, $\{x_6, x_7\}$	1.300
r_3	7	$\{x_1\}$, $\{x_2, x_5\}$, $\{x_4, x_5\}$, $\{x_2, x_3, x_6\}$, $\{x_2, x_6, x_7\}$, $\{x_3, x_4, x_6\}$, $\{x_4, x_6, x_7\}$	2.600
r_4	12	$\{x_1, x_3\}$, $\{x_1, x_4\}$, $\{x_1, x_6\}$, $\{x_1, x_7\}$, $\{x_4, x_5\}$, $\{x_2, x_3, x_5\}$, $\{x_2, x_3, x_6\}$, $\{x_2, x_5, x_6\}$, $\{x_2, x_5, x_7\}$, $\{x_2, x_6, x_7\}$, $\{x_3, x_4, x_6\}$, $\{x_4, x_6, x_7\}$	4.100
r_5	10	$\{x_1, x_3\}$, $\{x_1, x_4\}$, $\{x_1, x_6\}$, $\{x_1, x_7\}$, $\{x_4, x_5\}$, $\{x_2, x_3, x_6\}$, $\{x_2, x_5, x_6\}$, $\{x_2, x_6, x_7\}$, $\{x_3, x_4, x_6\}$, $\{x_4, x_6, x_7\}$	5.400
r_6	8	$\{x_1, x_3\}$, $\{x_1, x_4\}$, $\{x_1, x_6\}$, $\{x_1, x_7\}$, $\{x_4, x_5\}$, $\{x_2, x_6, x_7\}$, $\{x_3, x_4, x_6\}$, $\{x_4, x_6, x_7\}$	6.900

Tabla 2: YYC sobre la matriz A: testores típicos de las filas procesadas y tiempo de ejecución acumulado después de cada fila (mediana de 10 000 ejecuciones).

- El tiempo total, del orden de 7 μs, muestra que para matrices pequeñas el costo de YYC es despreciable. Su complejidad depende, sobre todo, del tamaño de Ψ^* en los pasos intermedios, que puede crecer exponencialmente con el número de columnas en matrices de baja densidad; por esto la implementación incluye un límite de seguridad sobre $|\Psi^*|$.

5. Ejercicio 3: algoritmo BT

5.1. El algoritmo BT

BT fue propuesto por Ruiz-Shulcloper, Bravo y Águila [4]. A diferencia de YYC, que construye los testores típicos a partir de la estructura interna de la matriz, BT es un algoritmo de *escala exterior*: recorre los subconjuntos de rasgos en un orden total fijo y, según el resultado de analizar cada uno, *salta* bloques completos de subconjuntos que no pueden ser testores típicos [5].

Codificación y orden. Cada subconjunto $T \subseteq R$ se representa por su n-uplo característico $\alpha = (a_1, \ldots, a_n) \in \{0,1\}^n$, con $a_j = 1$ si y sólo si $x_j \in T$; escribimos T_α para el conjunto asociado a α. El n-uplo se lee como un número natural en binario, con a_1 como dígito más significativo:

$$\nu(\alpha) = \sum_{j=1}^{n} a_j \, 2^{\,n-j}.$$

BT analiza los n-uplos en orden creciente de ν, desde $(0, \ldots, 0, 1) = \{x_n\}$ hasta $(1, \ldots, 1) = R$. Para $n = 3$ el orden es $001 = \{x_3\}$, $010 = \{x_2\}$, $011 = \{x_2, x_3\}$, $100 = \{x_1\}$, 101, 110, 111 (es el mismo que presenta [5, Tabla 4.1]).

Usaremos repetidamente el siguiente hecho elemental: si dos números $\alpha < \alpha'$ coinciden en sus primeros $k-1$ dígitos, todo β con $\alpha < \beta < \alpha'$ también coincide con ellos en esos dígitos (los números con un prefijo fijo forman un intervalo).

Proposición 3 (Salto S1: α no es testor)**.** *Sea α tal que T_α no es testor, sea r_i una fila con $r_i \cap T_\alpha = \emptyset$ y sea $k = \max\{j : r_i[j] = 1\}$ la posición del* último 1 *de r_i. Sea $\alpha' = (a_1, \ldots, a_{k-1}, 1, 0, \ldots, 0)$. Entonces $\alpha < \alpha'$ y ningún β con $\alpha < \beta < \alpha'$ es testor.*

Demostración. Como $r_i[k] = 1$ y r_i no corta a T_α, se tiene $a_k = 0$; luego α y α' coinciden en las posiciones $1, \ldots, k-1$ y en la posición k valen 0 y 1, así que $\alpha < \alpha'$. Sea $\alpha < \beta < \alpha'$. Por el hecho anterior, $b_j = a_j$ para $j < k$, y como $\beta < \alpha'$ con el mismo prefijo, $b_k = 0 = a_k$. Las posiciones $j > k$ no importan porque allí $r_i[j] = 0$. Entonces $r_i \cap T_\beta = r_i \cap T_\alpha = \emptyset$: la fila r_i sigue sin cubrirse y T_β no es testor. □

Proposición 4 (Salto S2: α es testor)**.** *Sea α tal que T_α es testor y sea $k = \max\{j : a_j = 1\}$ la posición del último 1 de α. Todo β con $\nu(\alpha) < \nu(\beta) < \nu(\alpha) + 2^{n-k}$ cumple $T_\beta \supset T_\alpha$ (inclusión propia); en particular, T_β es testor pero* no *es típico.*

Demostración. Como $a_j = 0$ para $j > k$, $\nu(\alpha)$ es múltiplo de 2^{n-k} y el intervalo $[\nu(\alpha), \nu(\alpha) + 2^{n-k})$ está formado exactamente por los n-uplos que coinciden con α en las posiciones $1, \ldots, k$, con cualquier valor en las posiciones $k+1, \ldots, n$. Si $\beta \neq \alpha$ está en ese intervalo, tiene algún $b_j = 1$ con $j > k$, luego $T_\beta \supset T_\alpha$ y $T_\beta \neq T_\alpha$. Todo superconjunto de un testor es testor, y no es típico porque contiene propiamente al testor T_α. □

El siguiente candidato tras S2 es $\nu(\alpha) + 2^{n-k}$: se "suma 1 en la posición del último 1" con acarreo hacia la izquierda. Si el acarreo se sale por la posición 1 ($\nu(\alpha) + 2^{n-k} \geq 2^n$), todos los n-uplos restantes son superconjuntos de T_α y el algoritmo termina. El algoritmo completo es:

1. $\alpha \leftarrow (0, \ldots, 0, 1)$.
2. Buscar la primera fila r_i (en el orden de la matriz) con $r_i \cap T_\alpha = \emptyset$.
3. **Si existe**, T_α no es testor: $k \leftarrow$ último 1 de r_i; $\alpha \leftarrow (a_1, \ldots, a_{k-1}, 1, 0, \ldots, 0)$ y volver a 2.
4. **Si no existe**, T_α es testor: registrarlo y decidir si es típico; $k \leftarrow$ último 1 de α; $\alpha \leftarrow \alpha + 2^{n-k}$. Si hubo desbordamiento, terminar; si no, volver a 2.

Proposición 5 (Corrección de BT)**.** *BT termina y analiza todos los testores típicos de la matriz.*

Demostración. En cada paso $\nu(\alpha)$ crece estrictamente y está acotado por $2^n - 1$, así que BT termina tras a lo sumo $2^n - 1$ pasos. Los únicos n-uplos que no se analizan son los que se saltan en S1 (no son testores, Proposición 3), los que se saltan en S2 y los que quedan tras el desbordamiento final (superconjuntos propios de un testor, Proposición 4). Ninguno de ellos es testor típico. □

Identificación de los testores típicos. BT, por sí solo, encuentra testores, y entre ellos hay testores no típicos (los superconjuntos de un testor que no son "consecutivos" a él en el orden). El enunciado pide un procedimiento para identificar los típicos; implementamos dos, que se comprueban entre sí:

1. **Durante la ejecución.** Cuando T_α resulta testor, se comprueba si cada rasgo de T_α tiene una fila típica; por la Proposición 1, esto equivale a que T_α sea típico.
2. **A posteriori.** Al terminar, se toman los elementos minimales (respecto de $\subseteq$) de la familia $\mathcal{F}$ de testores encontrados.

Proposición 6. *Los testores típicos de la matriz son exactamente los elementos minimales de $\mathcal{F}$.*

Demostración. Si T es típico, $T \in \mathcal{F}$ (Proposición 5) y ningún subconjunto propio $T' \subset T$ es testor, así que T es minimal en $\mathcal{F}$. Recíprocamente, sea T minimal en $\mathcal{F}$. Quitando rasgos a T mientras siga siendo testor se llega a un testor típico $T_0 \subseteq T$, que también está en $\mathcal{F}$; por minimalidad, $T_0 = T$. □

Orden de las filas. BT usa la *primera* fila de ceros, de modo que el orden de las filas no cambia el resultado pero sí el tamaño de los saltos S1. [5] menciona una variante que reordena previamente la matriz básica para provocar saltos mayores. Nuestra implementación *no* reordena nada, porque el Ejercicio 5 pide justamente medir el efecto de ordenar las filas.

5.2. Ejercicio 3.1: ejecución manual de BT sobre A

En la notación de n-uplos, las filas de A y la posición de su último 1 son

	r_1	r_2	r_3	r_4	r_5	r_6
fila	1010101	1000110	1101000	0011011	1001010	1001001
último 1	x_7	x_6	x_4	x_7	x_6	x_7

Para cada candidato α se recorren las filas $r_1, \ldots, r_6$ buscando la primera con ceros en todas las columnas de T_α. Detallamos los primeros pasos y los más ilustrativos; la Tabla 3 contiene el recorrido completo, con la justificación de cada decisión.

Paso 1. $\alpha = 0000001$, $T_\alpha = \{x_7\}$. r_1 contiene a x_7, pero $r_2 = \{x_1, x_5, x_6\}$ no: no es testor. El último 1 de r_2 está en x_6 (S1): $\alpha' = 0000010$.

Paso 2. $\{x_6\}$: r_1 no contiene a x_6. Último 1 de r_1: x_7, $\alpha' = 0000011$.

Paso 3. $\{x_6, x_7\}$ cubre r_1 y r_2, pero no $r_3 = \{x_1, x_2, x_4\}$. Último 1 de r_3: x_4, así que $\alpha' = 0001000$. Se saltan los 4 n-uplos $0000100, \ldots, 0000111$: todos tienen ceros en x_1, x_2, x_4 y no cubren r_3.

Pasos 4–6. $\{x_4\}$ no cubre r_1 (salto a 0001001); $\{x_4, x_7\}$ no cubre r_2 (salto a 0001010); $\{x_4, x_6\}$ no cubre r_1 (salto a 0001011).

Paso 7. $\{x_4, x_6, x_7\}$: $r_1 \to x_7$, $r_2 \to x_6$, $r_3 \to x_4$, $r_4 \to \{x_4, x_6, x_7\}$, $r_5 \to \{x_4, x_6\}$, $r_6 \to \{x_4, x_7\}$. Ninguna intersección es vacía: **es testor**. Además r_3, r_2 y r_1 son filas típicas de x_4, x_6 y x_7: **es típico**. El último 1 está en x_7 (S2): $\alpha' = 0001011 + 1 = 0001100$.

Paso 8. $\{x_4, x_5\}$: $r_1, r_2 \to x_5$ y $r_3, \ldots, r_6 \to x_4$; es testor y típico (x_4:r_3, x_5:r_1). El último 1 está en x_5: $\alpha' = 0001100 + 0000100 = 0010000$. Se saltan 0001101, 0001110 y 0001111, que son $\{x_4, x_5\}$ más x_6 y/o x_7.

Paso 13. $\{x_3, x_4, x_5\}$ es testor pero **no típico**: x_3 sólo aparece en r_1 (junto a x_5) y en r_4 (junto a x_4), así que no tiene fila típica. En efecto, contiene al testor $\{x_4, x_5\}$ encontrado en el paso 8, pero no es consecutivo a él en el orden y por eso S2 no lo evitó.

Pasos 42–43. $\{x_1, x_4\}$ y $\{x_1, x_3\}$ son típicos y producen los saltos más grandes: $1001000 + 0001000 = 1010000$ (7 superconjuntos omitidos) y $1010000 + 0010000 = 1100000$ (15 omitidos).

Paso 51. $\{x_1, x_2, x_3\}$ es testor no típico (x_2 sólo aparece en r_3, que también contiene a x_1). Al sumar 1 en la posición de x_3 el acarreo se sale por la izquierda ($1110000 + 0010000 = 10000000$): los 15 n-uplos restantes son superconjuntos de $\{x_1, x_2, x_3\}$ y **el algoritmo termina**.

Resumen de la ejecución.

- BT analizó **51** de los $2^7 - 1 = 127$ subconjuntos no vacíos y saltó los otros 76: 27 candidatos no fueron testores (salto S1) y 24 sí lo fueron (salto S2).
- De los 24 testores, **8 son típicos**: $\{x_4, x_6, x_7\}$, $\{x_4, x_5\}$, $\{x_3, x_4, x_6\}$, $\{x_2, x_6, x_7\}$, $\{x_1, x_7\}$, $\{x_1, x_6\}$, $\{x_1, x_4\}$, $\{x_1, x_3\}$ (en el orden en que BT los encuentra). Es exactamente el conjunto obtenido con YYC en la Sección 4.3. Los 16 testores restantes no son típicos, y los elementos minimales de la familia de 24 testores son, de nuevo, esos 8 (Proposición 6).
- La debilidad de BT señalada en [5] se ve claramente: dos tercios de los testores analizados son superconjuntos de testores típicos (por ejemplo $\{x_2, x_4, x_5\} \supset \{x_4, x_5\}$) que el salto S2 no puede evitar, porque no son consecutivos al testor en el orden binario. Como comparación, YYC probó 35 extensiones y nunca generó un testor no típico.

5.3. Ejercicio 3.2: implementación computacional de BT

Diseño. BT vive en la capa de dominio (`domain/bt.hpp`). Los saltos son dos funciones puras: `btJumpNotTestor` (S1: poner 1 en la posición k y ceros a su derecha) y `btJumpTestor` (S2: sumar 1 en la posición del último 1 con acarreo; devuelve "vacío" si hay desbordamiento). `BtSolver::solve` repite: buscar la primera fila de ceros con operaciones `AND` sobre palabras de 64 bits; si no hay, registrar el testor, decidir su tipicidad (procedimiento 1) y aplicar S2; si la hay, aplicar S1. El resultado contiene la lista de testores encontrados (en el orden de BT y marcados como típicos o no), los testores típicos y, opcionalmente, la traza de cada paso. El caso de uso `runBt` lee el reloj monótono antes y después de `solve`, de modo que el **tiempo total** medido no incluye la impresión; fuera de la medición aplica el procedimiento 2 (minimales) y comprueba que coincide con el 1. Como con YYC, `benchmarkBt` repite la ejecución y reporta la mediana. Hay cotas de seguridad sobre la cantidad de testores y de pasos de traza guardados.

Paso	α	T_α	Decisión	Justificación	Salto	α'
1	0000001	$\{x_7\}$	no testor	$r_2 \cap T_\alpha = \emptyset$	S1: x_6	0000010
2	0000010	$\{x_6\}$	no testor	$r_1 \cap T_\alpha = \emptyset$	S1: x_7	0000011
3	0000011	$\{x_6, x_7\}$	no testor	$r_3 \cap T_\alpha = \emptyset$	S1: x_4	0001000
4	0001000	$\{x_4\}$	no testor	$r_1 \cap T_\alpha = \emptyset$	S1: x_7	0001001
5	0001001	$\{x_4, x_7\}$	no testor	$r_2 \cap T_\alpha = \emptyset$	S1: x_6	0001010
6	0001010	$\{x_4, x_6\}$	no testor	$r_1 \cap T_\alpha = \emptyset$	S1: x_7	0001011
7	0001011	$\{x_4, x_6, x_7\}$	**típico**	$x_4{:}r_3,\ x_6{:}r_2,\ x_7{:}r_1$	S2: x_7	0001100
8	0001100	$\{x_4, x_5\}$	**típico**	$x_4{:}r_3,\ x_5{:}r_1$	S2: x_5	0010000
9	0010000	$\{x_3\}$	no testor	$r_2 \cap T_\alpha = \emptyset$	S1: x_6	0010010
10	0010010	$\{x_3, x_6\}$	no testor	$r_3 \cap T_\alpha = \emptyset$	S1: x_4	0011000
11	0011000	$\{x_3, x_4\}$	no testor	$r_2 \cap T_\alpha = \emptyset$	S1: x_6	0011010
12	0011010	$\{x_3, x_4, x_6\}$	**típico**	$x_3{:}r_1,\ x_4{:}r_3,\ x_6{:}r_2$	S2: x_6	0011100
13	0011100	$\{x_3, x_4, x_5\}$	no típico	x_3 sin fila típica	S2: x_5	0100000
14	0100000	$\{x_2\}$	no testor	$r_1 \cap T_\alpha = \emptyset$	S1: x_7	0100001
15	0100001	$\{x_2, x_7\}$	no testor	$r_2 \cap T_\alpha = \emptyset$	S1: x_6	0100010
16	0100010	$\{x_2, x_6\}$	no testor	$r_1 \cap T_\alpha = \emptyset$	S1: x_7	0100011
17	0100011	$\{x_2, x_6, x_7\}$	**típico**	$x_2{:}r_3,\ x_6{:}r_2,\ x_7{:}r_1$	S2: x_7	0100100
18	0100100	$\{x_2, x_5\}$	no testor	$r_4 \cap T_\alpha = \emptyset$	S1: x_7	0100101
19	0100101	$\{x_2, x_5, x_7\}$	no testor	$r_5 \cap T_\alpha = \emptyset$	S1: x_6	0100110
20	0100110	$\{x_2, x_5, x_6\}$	no testor	$r_6 \cap T_\alpha = \emptyset$	S1: x_7	0100111
21	0100111	$\{x_2, x_5, x_6, x_7\}$	no típico	x_5 sin fila típica	S2: x_7	0101000
22	0101000	$\{x_2, x_4\}$	no testor	$r_1 \cap T_\alpha = \emptyset$	S1: x_7	0101001
23	0101001	$\{x_2, x_4, x_7\}$	no testor	$r_2 \cap T_\alpha = \emptyset$	S1: x_6	0101010
24	0101010	$\{x_2, x_4, x_6\}$	no testor	$r_1 \cap T_\alpha = \emptyset$	S1: x_7	0101011
25	0101011	$\{x_2, x_4, x_6, x_7\}$	no típico	x_2 sin fila típica	S2: x_7	0101100
26	0101100	$\{x_2, x_4, x_5\}$	no típico	x_2 sin fila típica	S2: x_5	0110000
27	0110000	$\{x_2, x_3\}$	no testor	$r_2 \cap T_\alpha = \emptyset$	S1: x_6	0110010
28	0110010	$\{x_2, x_3, x_6\}$	no testor	$r_6 \cap T_\alpha = \emptyset$	S1: x_7	0110011
29	0110011	$\{x_2, x_3, x_6, x_7\}$	no típico	x_3 sin fila típica	S2: x_7	0110100
30	0110100	$\{x_2, x_3, x_5\}$	no testor	$r_5 \cap T_\alpha = \emptyset$	S1: x_6	0110110
31	0110110	$\{x_2, x_3, x_5, x_6\}$	no testor	$r_6 \cap T_\alpha = \emptyset$	S1: x_7	0110111
32	0110111	$\{x_2, x_3, x_5, x_6, x_7\}$	no típico	x_3 sin fila típica	S2: x_7	0111000
33	0111000	$\{x_2, x_3, x_4\}$	no testor	$r_2 \cap T_\alpha = \emptyset$	S1: x_6	0111010
34	0111010	$\{x_2, x_3, x_4, x_6\}$	no típico	x_2 sin fila típica	S2: x_6	0111100
35	0111100	$\{x_2, x_3, x_4, x_5\}$	no típico	x_2 sin fila típica	S2: x_5	1000000
36	1000000	$\{x_1\}$	no testor	$r_4 \cap T_\alpha = \emptyset$	S1: x_7	1000001
37	1000001	$\{x_1, x_7\}$	**típico**	$x_1{:}r_2,\ x_7{:}r_4$	S2: x_7	1000010
38	1000010	$\{x_1, x_6\}$	**típico**	$x_1{:}r_1,\ x_6{:}r_4$	S2: x_6	1000100
39	1000100	$\{x_1, x_5\}$	no testor	$r_4 \cap T_\alpha = \emptyset$	S1: x_7	1000101
40	1000101	$\{x_1, x_5, x_7\}$	no típico	x_5 sin fila típica	S2: x_7	1000110
41	1000110	$\{x_1, x_5, x_6\}$	no típico	x_5 sin fila típica	S2: x_6	1001000
42	1001000	$\{x_1, x_4\}$	**típico**	$x_1{:}r_1,\ x_4{:}r_4$	S2: x_4	1010000
43	1010000	$\{x_1, x_3\}$	**típico**	$x_1{:}r_2,\ x_3{:}r_4$	S2: x_3	1100000
44	1100000	$\{x_1, x_2\}$	no testor	$r_4 \cap T_\alpha = \emptyset$	S1: x_7	1100001
45	1100001	$\{x_1, x_2, x_7\}$	no típico	x_2 sin fila típica	S2: x_7	1100010
46	1100010	$\{x_1, x_2, x_6\}$	no típico	x_2 sin fila típica	S2: x_6	1100100
47	1100100	$\{x_1, x_2, x_5\}$	no testor	$r_4 \cap T_\alpha = \emptyset$	S1: x_7	1100101
48	1100101	$\{x_1, x_2, x_5, x_7\}$	no típico	x_2 sin fila típica	S2: x_7	1100110
49	1100110	$\{x_1, x_2, x_5, x_6\}$	no típico	x_2 sin fila típica	S2: x_6	1101000
50	1101000	$\{x_1, x_2, x_4\}$	no típico	x_2 sin fila típica	S2: x_4	1110000
51	1110000	$\{x_1, x_2, x_3\}$	no típico	x_2 sin fila típica	S2: x_3	fin

Tabla 3: Recorrido completo de BT sobre A. "S1: x_k" indica el salto por fila de ceros (1 en x_k y ceros a su derecha); "S2: x_k", el salto por testor (sumar 1 en la posición x_k). En los testores típicos se indica una fila típica de cada rasgo.

Interfaz de usuario. En el menú interactivo se añadieron las opciones 10 (ejecutar BT), 11 (BT paso a paso con explicación) y 12 (exportar el último resultado), que trabajan sobre la matriz actual (generada con el Ejercicio 1, cargada de archivo o ingresada a mano). Si la matriz no es básica, se reduce primero. También hay un subcomando:

```
testores bt --entrada data/matriz_A.txt          # testores, típicos, tiempo
testores bt --entrada data/matriz_A.txt --traza  # cada alfa y cada salto
testores bt --entrada data/matriz_A.txt --repeticiones 10000 \
            --csv resultados/bt_A.csv --latex resultados/bt_A_resumen.tex
```

Un fragmento de la salida:

```
Paso 13: alfa = 0011100 = {x3, x4, x5}
  ES testor (toda fila tiene un 1 en alguna columna de alfa) y NO es tipico:
  x3 no tiene fila tipica (se puede quitar y sigue siendo testor).
  Salto S2: ultimo 1 de alfa en x5 -> sumar 1 en esa posicion
  Siguiente: alfa' = 0100000 = {x2}  (se saltan 3 n-uplos)
...
Resultados de BT:
  n-uplos analizados: 51 de 127 posibles (2^7 - 1); se saltaron 76
  Testores encontrados: 24 (tipicos: 8, no tipicos: 16)
  Ambos procedimientos coinciden
```

Verificación de BT. Se añadieron 14 pruebas. Las principales:

- la traza exacta (n-uplos visitados, filas de ceros y saltos) sobre una matriz de 3×3 calculada a mano;
- BT coincide con el oráculo exhaustivo y con YYC en 500 matrices básicas aleatorias de 1 a 12 columnas;
- en 150 matrices aleatorias se recorren *todos* los n-uplos que BT no analizó y se comprueba que ninguno es testor típico, que el orden de visita es estrictamente creciente y que cada testor se clasificó correctamente; es una comprobación empírica de las Proposiciones 3 y 4;
- los minimales de los testores encontrados coinciden con los típicos (Proposición 6).

Además, la traza de la Tabla 3 se reprodujo con una implementación independiente de BT escrita en Python a partir de las Proposiciones 3 y 4: los 51 pasos coinciden.

5.4. Ejercicio 3.3: resultados sobre la matriz A

n-uplos analizados	51 de $2^7 - 1 = 127$
Testores encontrados	24 (8 típicos, 16 no típicos)
Testores típicos	$\{x_1, x_3\}$, $\{x_1, x_4\}$, $\{x_1, x_6\}$, $\{x_1, x_7\}$, $\{x_4, x_5\}$, $\{x_2, x_6, x_7\}$, $\{x_3, x_4, x_6\}$, $\{x_4, x_6, x_7\}$
Tiempo total (μs)	23.900

Tabla 4: BT sobre la matriz A: testores típicos obtenidos y tiempo total de ejecución (mediana de 10 000 ejecuciones). La lista completa de los 24 testores encontrados está en `docs/resultados/bt_A.csv`.

Interpretación.

- Los testores típicos coinciden exactamente con los de la ejecución manual y con los de YYC (Tabla 2).
- El tiempo total es de unas pocas decenas de microsegundos. Con mediciones hechas una tras otra en las mismas condiciones, BT tardó entre 24 y 32 μs y YYC unos 17 μs. (Los

valores absolutos cambian con el estado del equipo, por ejemplo con el modo de energía; por eso la tabla de YYC de la Sección 4.5, medida en otra sesión, muestra unos 7 μs. Sólo tiene sentido comparar tiempos medidos en la misma sesión.)

- El costo por candidato de ambos algoritmos es parecido. BT es más lento en A porque analiza más candidatos (51 frente a 35) y 16 de ellos son testores no típicos que YYC nunca genera. La comparación sistemática, incluido el efecto del orden de las filas sobre los saltos de BT, corresponde al Ejercicio 5.

Referencias

[1] E. Alba-Cabrera, J. Ibarra-Fiallo, S. Godoy-Calderón y F. Cervantes-Alonso. YYC: A fast performance incremental algorithm for finding typical testors. En *Progress in Pattern Recognition, Image Analysis, Computer Vision, and Applications (CIARP 2014)*, LNCS 8827, pp. 416–423. Springer, 2014.

[2] E. Alba-Cabrera, S. Godoy-Calderón y J. Ibarra-Fiallo. Generating synthetic test matrices as a benchmark for the computational behavior of typical testor-finding algorithms. *Pattern Recognition Letters*, 80:46–51, 2016.

[3] M. Lazo-Cortés, J. Ruiz-Shulcloper y E. Alba-Cabrera. An overview of the evolution of the concept of testor. *Pattern Recognition*, 34(4):753–762, 2001.

[4] J. Ruiz-Shulcloper, M. A. Bravo y F. L. Águila. Algoritmos BT y TB para el cálculo de todos los tests típicos. *Revista Ciencias Matemáticas*, 6(2), 1982.

[5] A. Pons-Porrata. *Desarrollo de algoritmos para la estructuración dinámica de información y su aplicación a la detección de sucesos.* Tesis doctoral, Universitat Jaume I, Castellón, 2004. Sección 4.2.2.

[6] M. Matsumoto y T. Nishimura. Mersenne Twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, 8(1):3–30, 1998.

[7] ISO/IEC 14882:2017. *Programming languages — C++*. International Organization for Standardization, 2017.